

ABAP Lock Concept

Adrian Streit, Johannes Rank, Borys Levkovskiy



Prerequisites

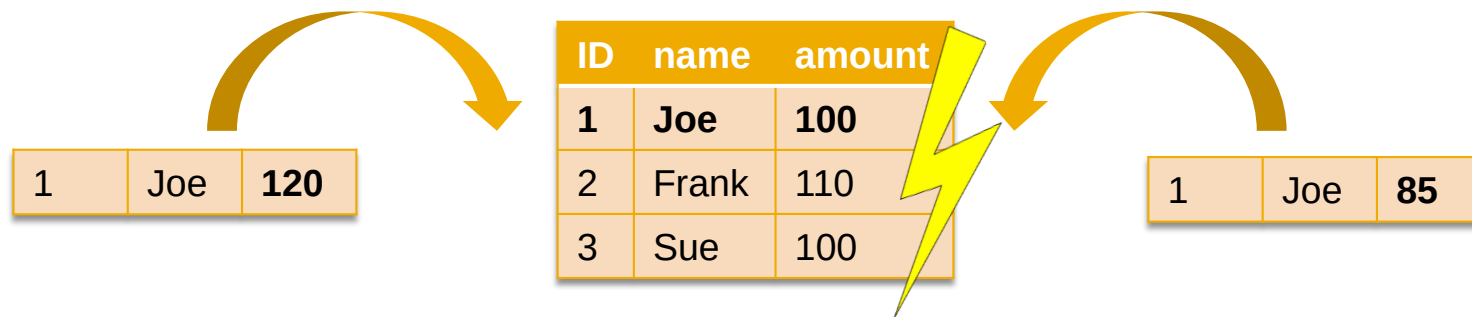
- Experience with ABAP Workbench and Eclipse
- Basic ABAP programming skills
- Experience with database access within ABAP programs

Agenda

- I. Introduction**
- II. Data consistency**
- III. Lock object types**
- IV. Lock usage**
- V. Creation process**
- VI. Enqueue/dequeue functions**
- VII. Monitoring**

Operational concurrency

- Using SAP applications we access **shared data resources** from the database
- Consistent data if we only perform
 - READ operations
 - READ and WRITE operations in sequence
- **Data inconsistency:** concurrent WRITE operations using same dataset



- Solution: **lock concept** to facilitate operational concurrency and data consistency together

General information

- What happens when several users want to work on the same data set? → data inconsistency
- Lock objects (SAP dictionary structure) prevent simultaneous changes on data set
- Beside the lock concept of the database, SAP implements its own lock concept
- SAP does not use the database's lock concept

Trigger for database commits and rollbacks

Implicit database commits



- Completion of a dialog step
- Transmission of error, warning and information messages
- Call of an remote function module
- Program interruption with instruction WAIT

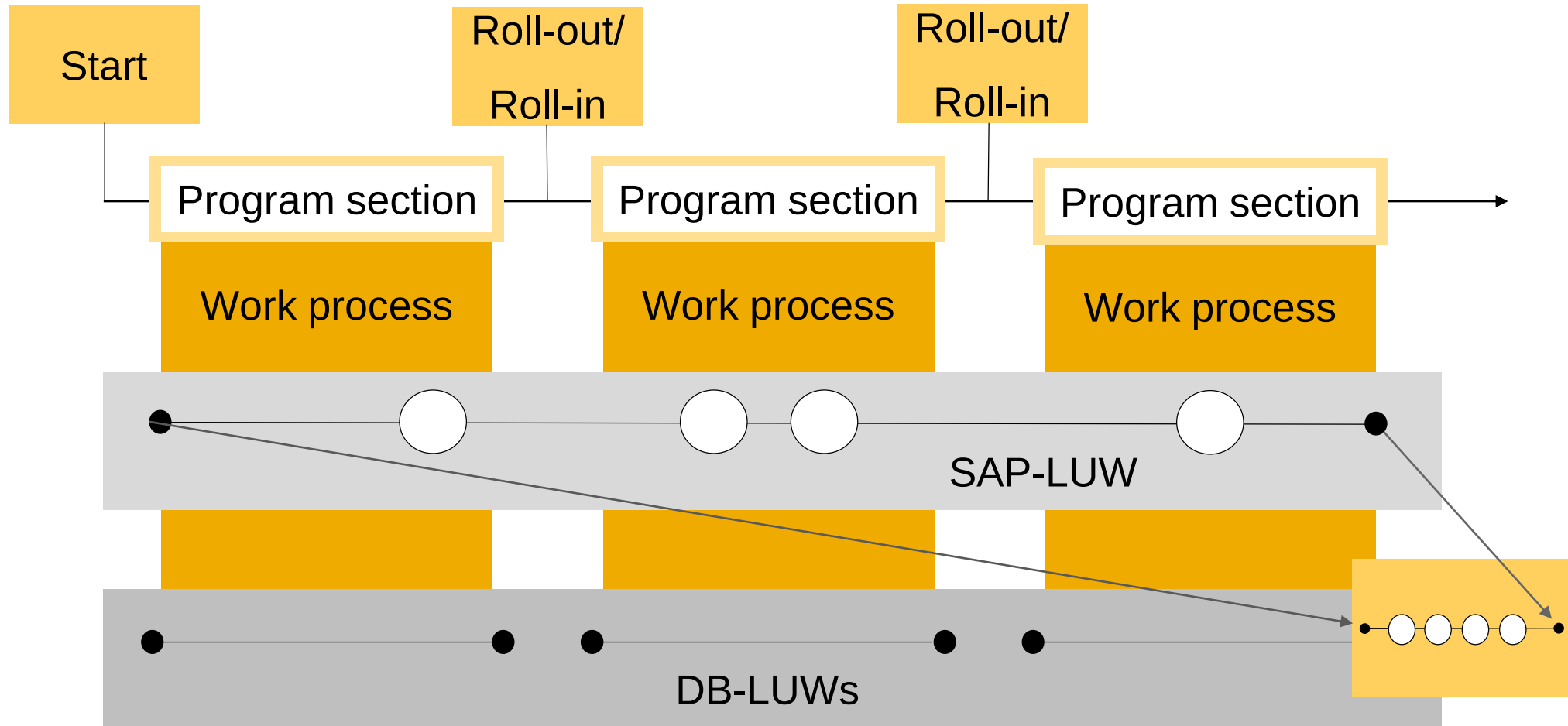
Implicit database rollbacks



- Occurrence of a runtime error
- Transmission of a termination or exit message

II. Data consistency

SAP LUW (Logical Unit of Work)



SAP LUW (Logical Unit of Work)

Problem

- New data sets are transferred into the database **by each screen**
- Only if last screen of the program is reached and all data sets are written to the database the data are correct

Solution

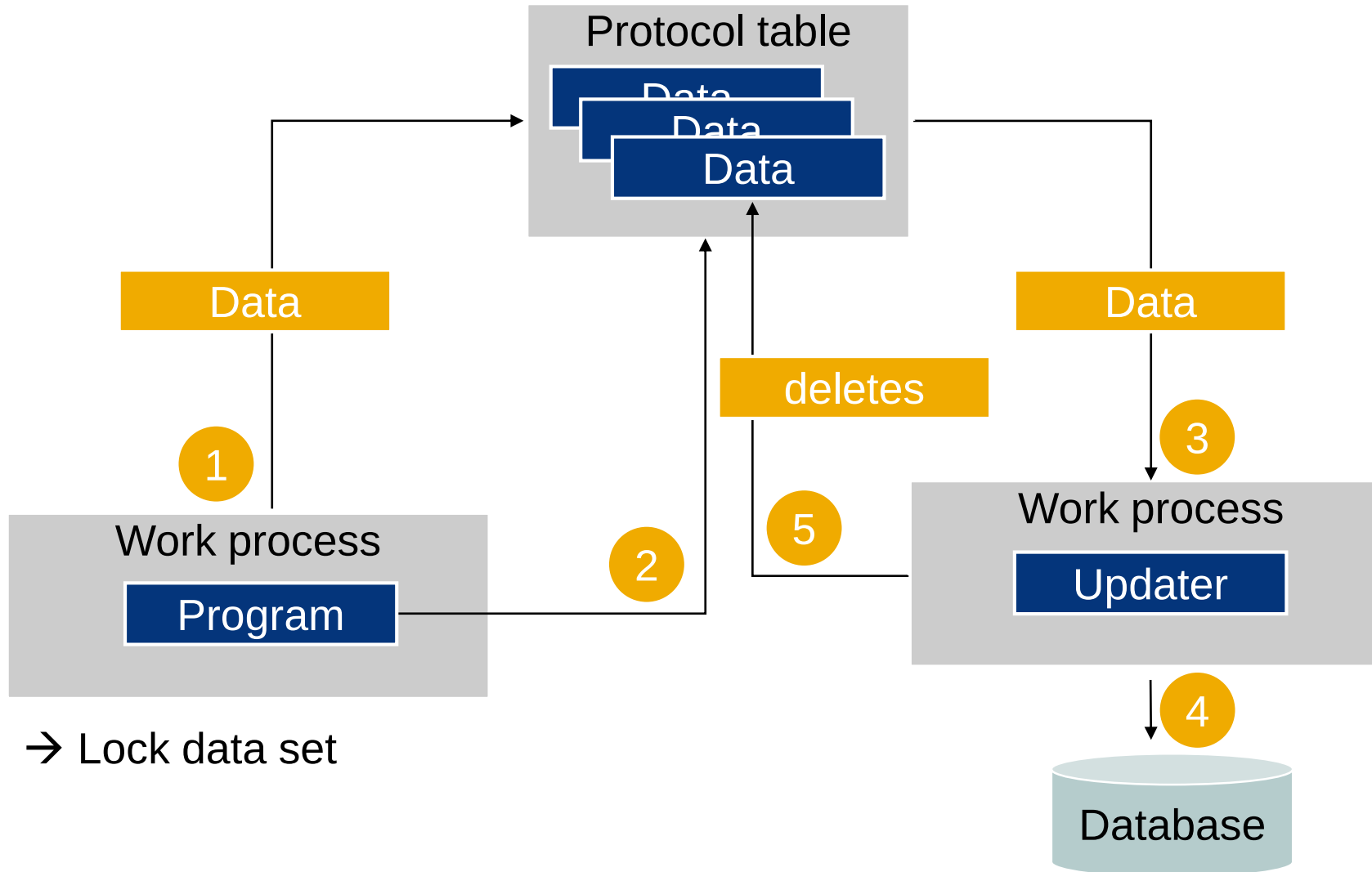
- Updates to database are written to the **updater process**
- Only when last screen of the program is reached, the updater receives a trigger to write data to database (commit) or to dismiss (rollback)

→ ensures **data integrity**

II. Data consistency

The updater process

- Schema:



Using the updater process

Function Modules

- **CALL FUNCTION ... IN UPDATE TASK.**

SAP LUW termination

- **COMMIT WORK [AND WAIT].**
- **ROLLBACK WORK.**

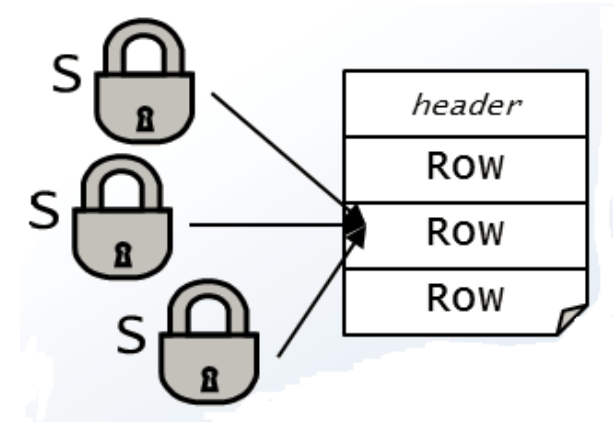
Registration of subroutines

- **PERFORM ... ON COMMIT | ROLLBACK.**

II. Data consistency

SAP lock concept

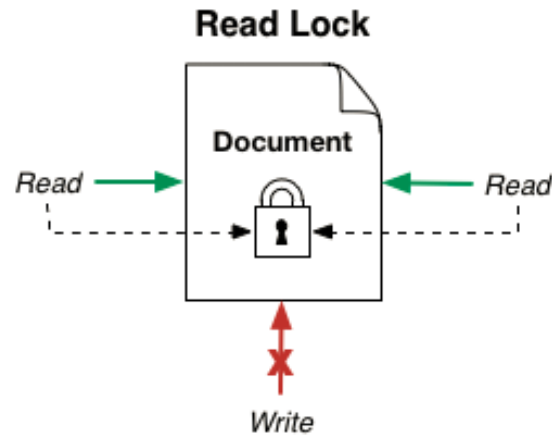
- Ensure **concurrent database access** from several SAP applications by guaranteeing data consistency at the same time
- SAP locks are **independent from database locks**
- SAP locks are claimed till the end of an SAP LUW
- No physical locks on database, but entries in **specific SAP table** which contains all SAP locks
- Specific **SAP lock objects** are required to make use of the SAP lock concept
- Lock objects have to **start with the letter E** (e.g. EZ_DEMOTABLE)



Shared vs. exclusive lock

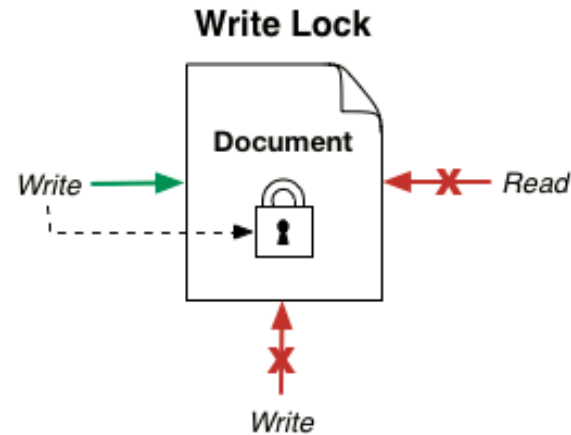
Read lock = shared lock

- Locks data set for reading
- Multiple shared locks on one data set at the same time possible
- Prevents the SAP system from creating an exclusive lock on a data set



Write lock = exclusive lock

- Locks data set for writing/changing
- No shared locks, only one is allowed at the same time
- First come, first serve



Enqueue and dequeue lock objects

- Two function modules are generated automatically when creating a lock object:



- Before writing/reading data set → lock data set
- After writing/reading data set → unlock data set
- In case of an error → unlock data set
- Locks can be displayed and deleted in transaction **SM12 (SAP GUI)**

IV. Lock usage

Acquire and Release

```
* Set exclusive lock on table <TABLENAME>
CALL FUNCTION 'ENQUEUE_EZ_W_MACHINE'
  EXPORTING
    mode_TABLENAME      = 'E'
    id                   = primaryKey
  EXCEPTIONS
    foreign_lock         = 1
    system_failure       = 2
    OTHERS                = 3.
IF sy-subrc <> 0.
  RAISE EXCEPTION TYPE exceptionclass EXPORTING message = 'Getting the lock failed.'.
ENDIF.
```

RELEASE existing lock object

ACQUIRE new lock object

```
* Release exclusive lock which is hold on table <TABLENAME>
CALL FUNCTION 'DEQUEUE_EZ_W_MACHINE'
  EXPORTING
    mode_TABLENAME      = 'E'
    id                   = primaryKey.

ENDFUNCTION.
```

Selection of enqueue parameters

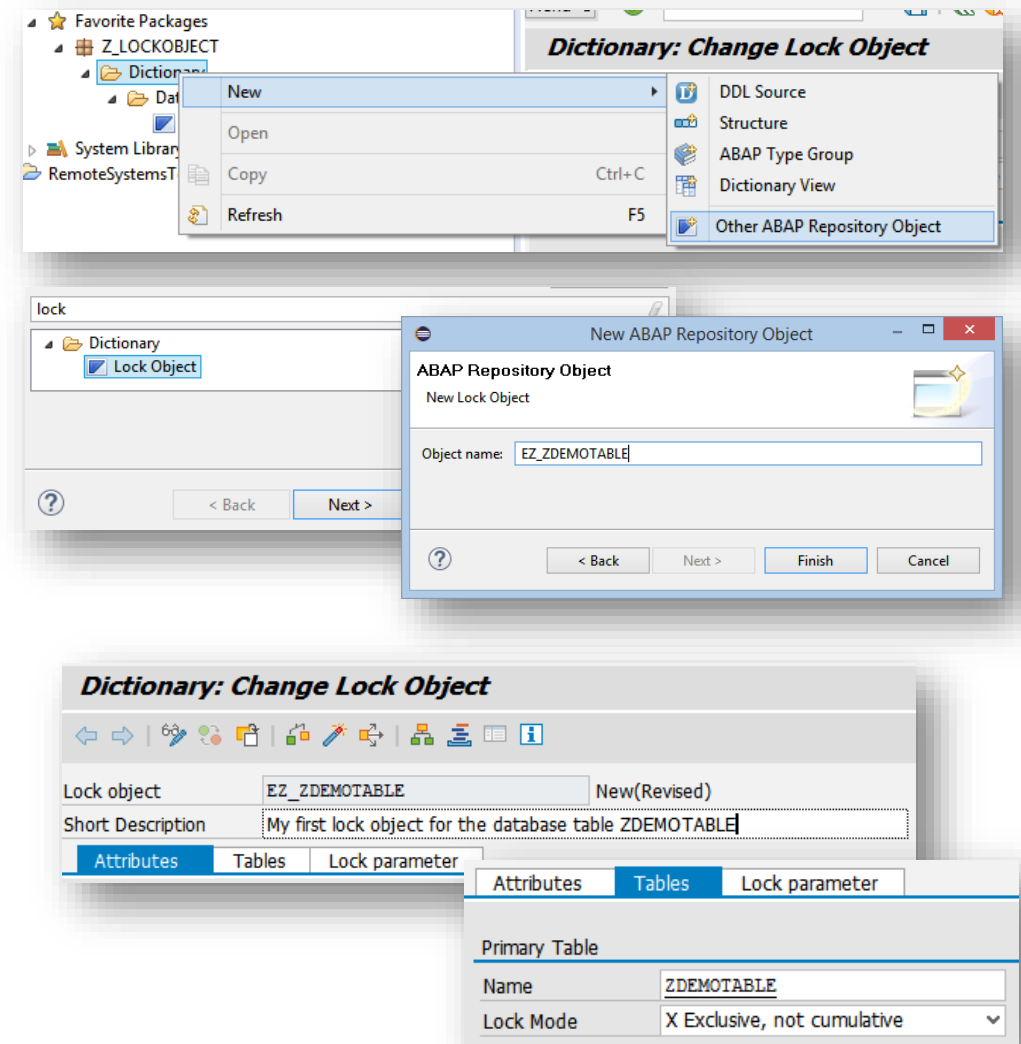
Parameter	Value	Meaning
mode_<tabname>	'S' 'E'	Read lock (shared) Exclusive lock (not shared)
<lock parameter>	<value>	Field values used to lock the data set
_wait	SPACE 'X'	If foreign lock, no new attempt If foreign lock, new attempt
_collect	SPACE 'X'	Lock without local lock container Lock with local lock container

V. Creation process

Create first lock object

Prerequisites: existing database table

- (1) Right-click on Dictionary → New → Other ABAP Repository Object
- (2) Choose entry “Lock Object” → Next → Define name (stick to the SAP and course conventions!) → Finish
- (3) Fill in Description
- (4) Tables tab: reference to the database table you want to create this lock object for → Save (choose the same package as the table is located in and create a new transport request) → Activate



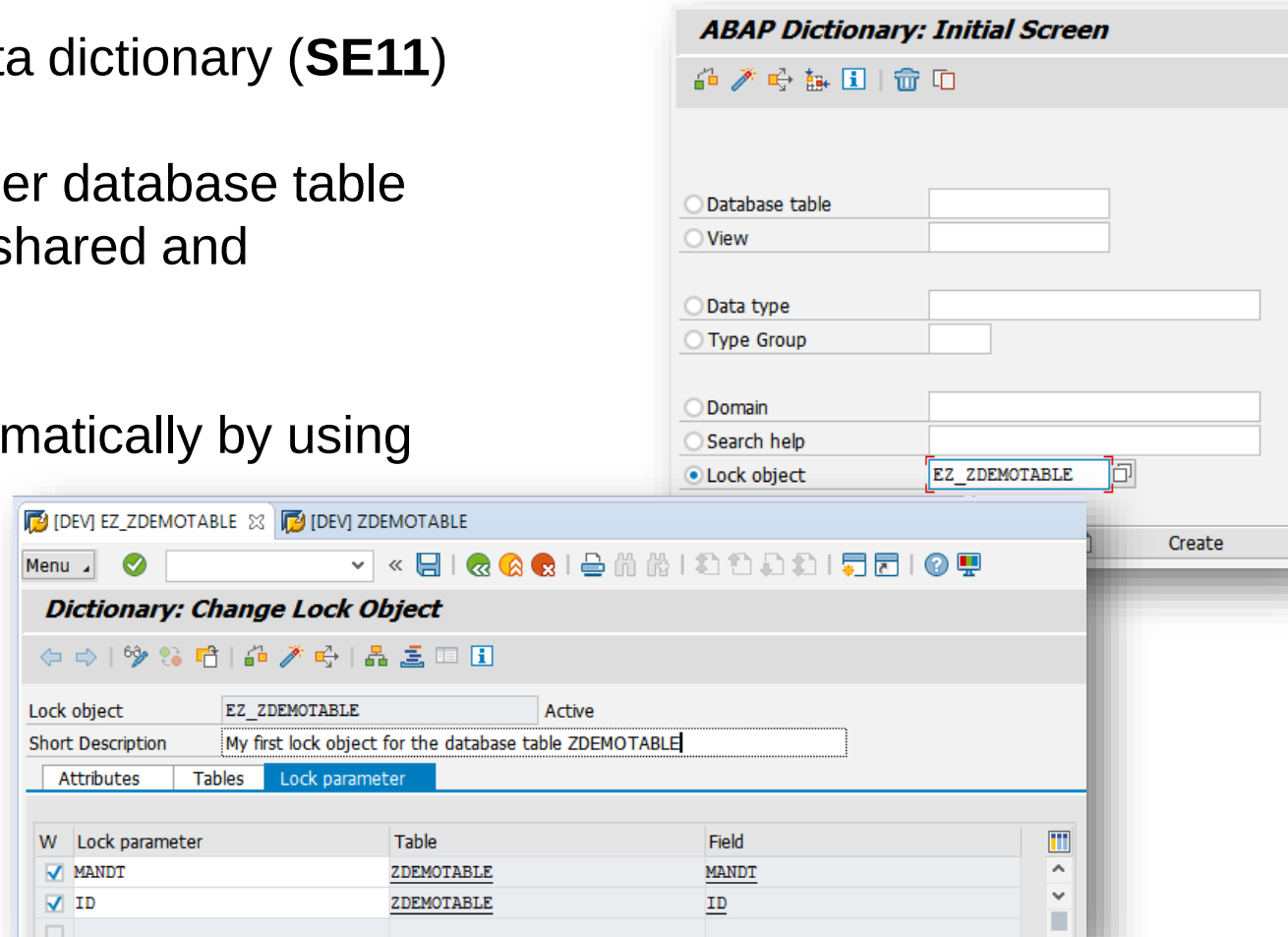
V. Creation process

Create first lock object

- Creation can also be done in data dictionary (**SE11**)
- There is **only one lock object** per database table which handles all type of locks (shared and exclusive)
- Lock objects are generated automatically by using the primary key

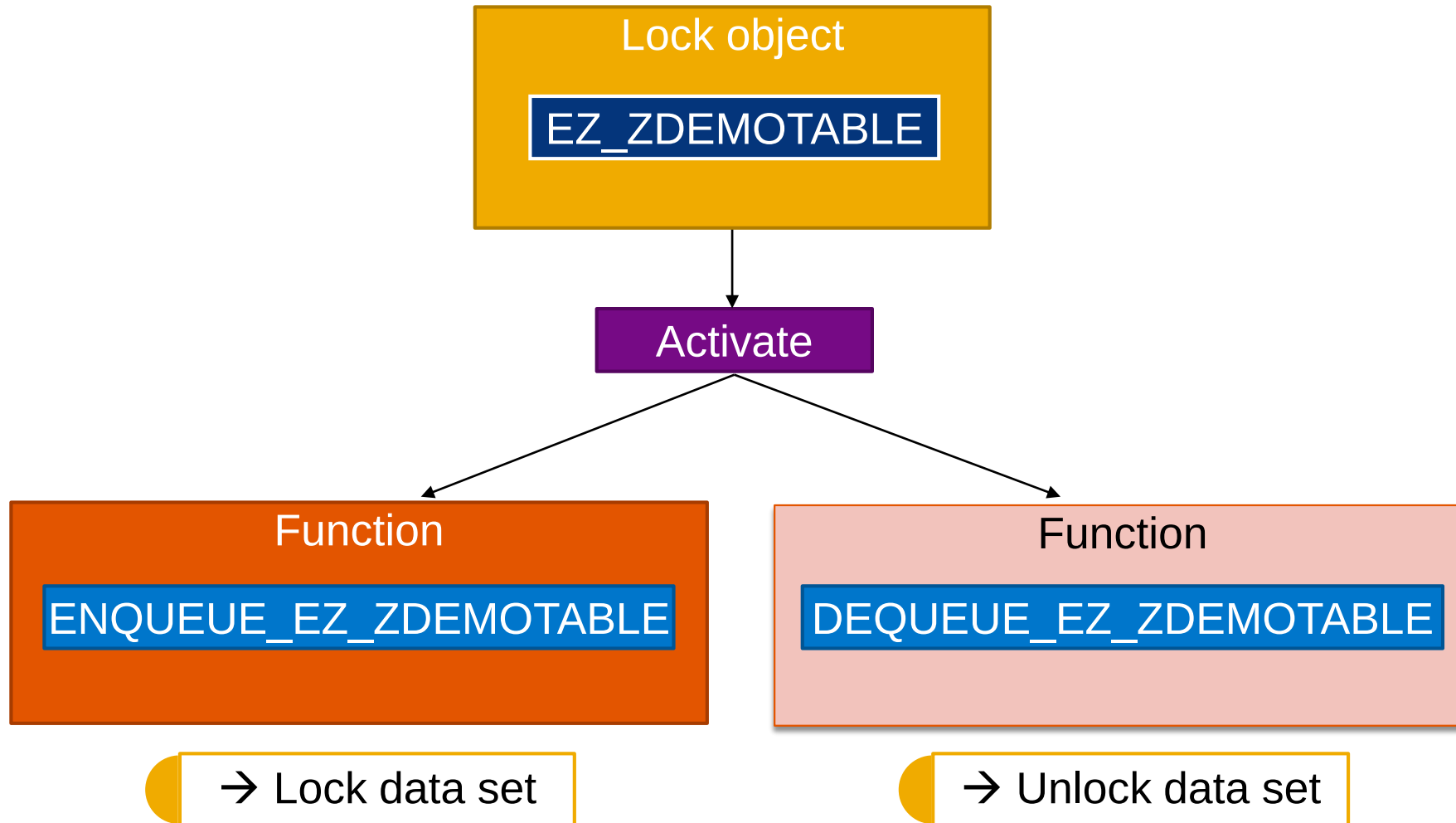
a) ENQUEUE_<Lock_Object>

b) DEQUEUE_<Lock_Object>



VI. Enqueue/dequeue functions

Automatic generation



VII. Monitoring

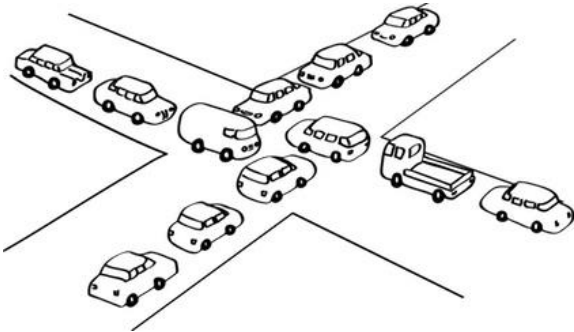
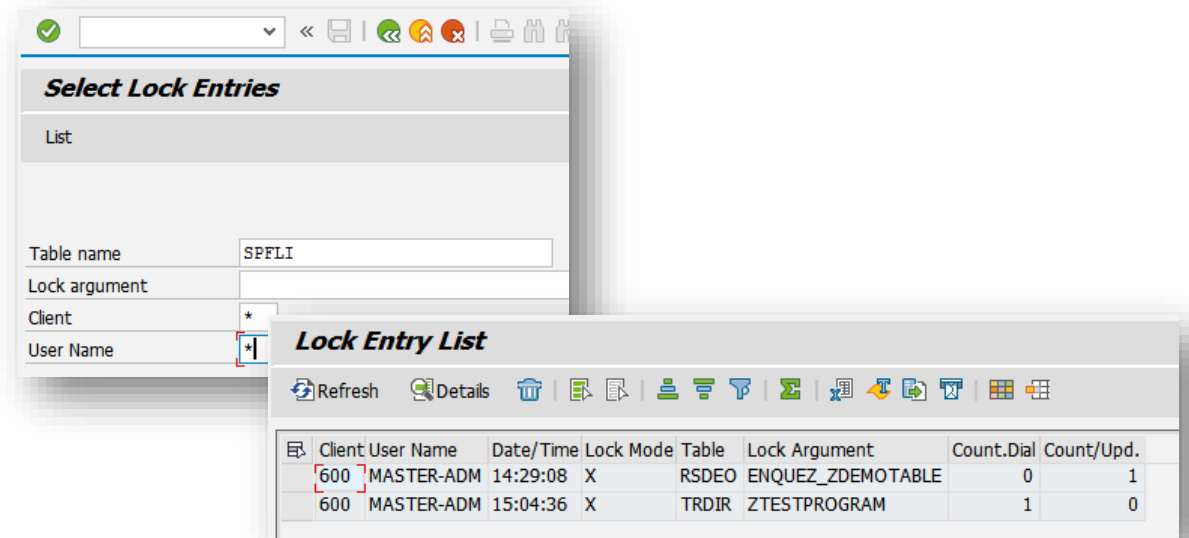
Display and deletion of current locks



Monitoring current locks is only available by using the **SAP GUI**

SM12

Transaction **SM12** enables you to **search** for current locks and to **delete** them



Please keep in mind to avoid deadlocks situations by acquiring dependent locks



Now you know how to apply SAP locks for your program.

To consolidate your knowledge, you can do task 1 and 2 of the Lock Concept exercise.



Check your knowledge

Check your knowledge

- Data inconsistency arises from concurrent operational access on the same dataset.
☐ True ☐ False
- Enqueue and dequeue functions are generated automatically by creating the corresponding SAP lock object.
☐ True ☐ False
- Explain the difference between database and SAP locks. What are their main objectives?
- Which of the following message types is implicitly used to perform a database commit?
☐ EXIT ☐ ERROR ☐ TERMINATION ☐ WARNING ☐ INFORMATION

Solution

Solution

- Data inconsistency arises from concurrent operational access on the same dataset.

☐ True ☒ False

- Enqueue and dequeue functions are generated automatically by creating the corresponding SAP lock object.

☒ True ☐ False

- Explain the difference between database and SAP locks. What are their main objectives?

See section *Data Consistency*

- Which of the following message types is implicitly used to perform a database commit?

☐ EXIT ☒ ERROR ☐ TERMINATION ☒ WARNING ☒ INFORMATION

References

- Schwaiger, R. (2016): Schrödinger programmiert ABAP: Das etwas andere Fachbuch, SAP PRESS, 2016
- Keller, H.; Krüger, S. (2006): ABAP Objects: ABAP-Programmierung mit SAP NetWeaver, Rheinwerk Publishing

© 2018 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. Please see <http://global12.sap.com/corporate-en/legal/copyright/index.epx> for additional trademark information and notices.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors.

National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP SE or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP SE or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

In particular, SAP SE or its affiliated companies have no obligation to pursue any course of business outlined in this document or any related presentation, or to develop or release any functionality mentioned therein. This document, or any related presentation, and SAP SE's or its affiliated companies' strategy and possible future developments, products, and/or platform directions and functionality are all subject to change and may be changed by SAP SE or its affiliated companies at any time for any reason without notice. The information in this document is not a commitment, promise, or legal obligation to deliver any material, code, or functionality. All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Readers are cautioned not to place undue reliance on these forward-looking statements, which speak only as of their dates, and they should not be relied upon in making purchasing decisions.